

MISSING VALUES

- First ask **why**: MCAR (random), MAR (depends on other variables), MNAR (depends on the missing value itself, e.g. high earners skip income).

Option	When
drop rows / columns	few missing, MCAR; column > ~60% empty
median / mode impute	simple, robust baseline
group-wise impute	missingness differs by segment
KNN / iterative (MICE)	MAR with strong correlates
add a <code>was_missing</code> flag	missingness itself is signal
tree models (XGB/LightGBM)	handle NaN natively

- Fit imputers on **training data only** (inside the pipeline).

OUTLIERS

- Detect: IQR fences (Q1 – 1.5·IQR, Q3 + 1.5·IQR), z-score > 3 (inflated by the outlier itself), robust z with median and MAD, Isolation Forest.
- Decide: error (fix/drop), real but extreme (keep). Treat: winsorise (cap at 1st/99th percentile), log-transform, robust scaler, robust models (trees, Huber).
- Never delete points only because they hurt the model.

ENCODING CATEGORICALS

Method	Use	Caution
one-hot	few levels, linear/NN	wide with many levels; drop one for linear
ordinal	natural order (S < M < L)	do not order unordered labels
frequency / count	high cardinality	collisions of equal counts
target / mean encoding	high cardinality	leaks: compute out-of-fold, smooth
hashing	huge, streaming	collisions
embeddings	very high cardinality, NN	needs data
native (CatBoost, LightGBM)	boosting	built-in handling

- Unseen categories at serving: `handle_unknown="ignore"`, an "other" bucket.

SCALING AND TRANSFORMS

Scaler	Formula	Use
StandardScaler	$(x - \text{mean}) / \text{SD}$	linear/logistic, SVM, KNN, PCA, NN
MinMaxScaler	$(x - \text{min}) / (\text{max} - \text{min})$	bounded inputs, NN, images
RobustScaler	$(x - \text{median}) / \text{IQR}$	outliers

- Trees, forests and boosting **do not need scaling**. Fit the scaler on train only.
- Log / log1p** for right-skewed positives (income, counts); Box-Cox / Yeo-Johnson for general skew; binning for non-linear thresholds (KBinsDiscretizer).

FEATURE IDEAS

- Dates**: hour, day of week, month, `is_weekend`, holiday, days since event; cyclic encoding ($\sin/\cos$ of $2\pi \cdot h/24$).
- Aggregates**: user mean/sum/count over 7/30/90 days, recency-frequency-monetary (RFM), ratios (spend per visit), trends (7-day vs 30-day).
- Interactions** (x1·x2), polynomials, domain features (BMI, distance), text length, TF-IDF, embeddings, geo distance.
- Window features** must use data **before** the prediction time only.
- Selection: **filter** (correlation, mutual information, chi-square, variance threshold), **wrapper** (recursive feature elimination), **embedded** (L1, tree importance, permutation importance). Remove constant, duplicate and leaky features; check stability over time.

FEATURES BY DATA TYPE

- Text**: length, counts, TF-IDF n-grams, embeddings; lower-case, strip, handle rare words.
- Time series**: lags, rolling mean/std, differences, expanding stats, calendar and holiday flags.
- Geo**: distance to points, clusters, region;
- Images**: pretrained embeddings.
- Counts/rates**: per-user averages, ratios with smoothing (add a prior) for small denominators.

- Target (mean) encoding recipe**: out-of-fold category means, smoothed: $(n \cdot \text{cat_mean} + m \cdot \text{global_mean}) / (n + m)$.

DIMENSIONALITY REDUCTION

- PCA**: centre and **scale**, take eigenvectors of the covariance matrix; PC1 has maximal variance; components are orthogonal linear mixes. Keep enough for 90-95% explained variance (`explained_variance_ratio_`, scree plot). Unsupervised; loses interpretability; helps noise, speed, multicollinearity.
- t-SNE / UMAP**: non-linear embeddings for 2-D visualisation of clusters; distances between clusters and cluster sizes are **not** meaningful; not for feeding models (UMAP is better for that).
- LDA**: supervised, finds axes that best separate classes (at most classes – 1). **Autoencoders**: non-linear compression. **SVD**: matrix factorisation behind PCA and recommenders.

```
from sklearn.compose import
ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler)
from sklearn.pipeline import
make_pipeline
pre = ColumnTransformer([
    ("num", make_pipeline(
        SimpleImputer(strategy="median"),
        StandardScaler()), num_cols),
    ("cat", OneHotEncoder(
        handle_unknown="ignore"), cat_cols)])
```

CONFUSION MATRIX AND CLASSIFICATION METRICS

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

Metric	Formula	Question answered
Accuracy	$(TP + TN) / \text{all}$	share correct (misleads when imbalanced)
Precision	$TP / (TP + FP)$	of flagged, how many are real
Recall (TPR, sensitivity)	$TP / (TP + FN)$	of real, how many found
Specificity	$TN / (TN + FP)$	of negatives, how many cleared
FPR	$FP / (FP + TN)$	false alarm rate
F1	$2PR / (P + R)$	harmonic mean; F β weights recall $\beta \times$ more
Balanced accuracy	$(TPR + TNR) / 2$	imbalance-aware accuracy
MCC	all four cells	single robust score

- Example: 100 flagged, 60 real, 120 real in total: precision 60%, recall 50%, F1 0.545.
- A model that always says "no fraud" on 99:1 data: accuracy 99%, recall 0%.
- Raise the threshold $\rightarrow$ precision up, recall down; lower it $\rightarrow$ the reverse.

WORKED EXAMPLE

1,000 cases, 50 fraud. Model flags 80: 40 are fraud (TP = 40), so FP = 40, FN = 10, TN = 910.

- Accuracy = $950 / 1000 = 95\%$. Precision = $40 / 80 = 50\%$. Recall = $40 / 50 = 80\%$. F1 = 0.615. FPR = $40 / 950 = 4.2\%$.
- A "never fraud" model: accuracy 95%, recall 0%. Same accuracy, useless.
- Cost view: FN costs \$500, FP costs \$5: cost = $10 \cdot 500 + 40 \cdot 5 = \$5,200$. Lowering the threshold trades FNs for cheap FPs until cost stops falling.

CURVES AND PROBABILITIES

- **ROC-AUC:** P(random positive ranks above random negative); 0.5 = chance, 1 = perfect. Threshold-free, ranking quality; looks optimistic with rare positives.
- **PR-AUC / average precision:** focuses on the positive class; baseline = prevalence. Prefer for imbalance.

- **Log loss** (cross-entropy) punishes confident wrong probabilities; **Brier score** = mean squared probability error. **Calibration:** of cases scored 0.7, ~70% are positive.
- **KS statistic** (max gap between cumulative good/bad), **Gini** = $2 \cdot \text{AUC} - 1$ (credit scoring), **lift/gain** at top decile.

CHOOSING THE METRIC

Situation	Metric
Missing a case is costly (cancer, fraud)	recall at acceptable precision
False alarms are costly (spam block, ad fatigue)	precision
Both matter	F1 or the PR curve
Known costs	expected cost = $c_{\text{FN}} \cdot \text{FN} + c_{\text{FP}} \cdot \text{FP}$, pick the threshold that minimises it
Limited review capacity (50 cases/day)	precision@K
Balanced, symmetric costs	accuracy / ROC-AUC
Probabilities are used (pricing, risk)	log loss, calibration

- Say what the metric means for the business, then **tune the threshold**, not only the model.

REGRESSION, RANKING, CLUSTERING

- **MAE** mean |error| (robust, in units). **MSE** mean squared (punishes big misses). **RMSE** $\sqrt{\text{MSE}}$ (units). Errors 1, 1, 1, 9: MAE 3, RMSE 4.6.
- $R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$ (can be negative out of sample). **MAPE** = mean |error|/|actual| (explodes near 0). **SMAPE**, **RMSLE** (relative errors).
- **Ranking:** precision@K, recall@K, MRR (1/rank of first hit), MAP, **NDCG** (graded relevance, discounts by log2 position).
- **Clustering:** silhouette (-1 to 1: closeness to own vs nearest cluster), Davies-Bouldin (lower better), inertia/elbow; with labels: adjusted Rand index, NMI. Judge usefulness by actionability.

IMBALANCED DATA PLAYBOOK

1. **Metrics first:** precision, recall, PR-AUC; never plain accuracy.
2. **Stratified** splits so every fold has positives.
3. **Class weights** (`class_weight="balanced"`, `scale_pos_weight`): the simplest fix.

4. **Resample training folds only:** undersample the majority, oversample the minority, **SMOTE** (synthetic neighbours), ADASYN.
5. **Tune the threshold** on a validation set; recalculate probabilities if resampled.
6. Try anomaly detection when positives are extremely rare; collect more positives; cost-sensitive learning.

TRAP Resampling before the split duplicates rows into validation and inflates scores. Do it inside the training fold (`imblearn.Pipeline`).

```
from sklearn.metrics import (
    precision_recall_curve,
    classification_report,
    average_precision_score)
p, r, t = precision_recall_curve(y,
    score)
print(classification_report(y, pred))
```

THRESHOLD AND CALIBRATION RECIPE

1. Get scores on a **validation** set (never the training set).
2. Compute precision/recall (or cost) for each threshold; plot the PR curve.
3. Choose the threshold that meets the business constraint (recall $\geq 80\%$, review capacity, minimum expected cost).
4. Fix it, then report the test-set metrics once at that threshold.
5. Re-check calibration after resampling or drift; recalibrate (Platt, isotonic).

METRIC TRAPS

TRAP Optimising AUC when only the top 1% is actioned; use precision@K. Comparing models by different thresholds or on different data. Reporting accuracy for imbalanced classes, R^2 as accuracy, or RMSE without a baseline (always compare with predicting the mean/last value). Tuning on the test set; macro vs micro vs weighted averaging in multiclass (macro treats classes equally).

SAY IT IN 20 SECONDS

- "Accuracy lies on imbalanced data; use precision, recall and PR-AUC."
- "Pick the threshold from the cost of each kind of error."
- "ROC-AUC is about ranking, not about calibrated probabilities."
- "Scale, impute and encode inside the pipeline, fit on train only."